

1. Project Structure & App Split

To maintain separation of concerns as the system scales, the monolithic architecture is split into five distinct domain apps and a core logic layer:

- **accounts**: User management and authentication.
- **facilities**: Physical infrastructure (Address, PracticeLocation, Hospital, DiagnosticCenter, Chamber) and their respective metadata.
- **doctors**: Practitioners, specialties, and affiliations.
- **tests**: Medical tests, categories, and facility-specific pricing.
- **bookings**: Booking records and statuses.
- **services/**: A top-level Python package for isolated business logic (pricing, availability, scheduling) independent of DRF request cycles.

2. Core Data Models

I. Accounts App (**accounts/models.py**)

Python

```
import uuid
from django.db import models
from django.contrib.auth.models import AbstractBaseUser, BaseUserManager,
PermissionsMixin
from django.utils import timezone
```

```
class UserManager(BaseUserManager):
    def create_user(self, phone_number, password=None, **extra_fields):
        if not phone_number:
            raise ValueError('The Phone Number must be set')
        user = self.model(phone_number=phone_number, **extra_fields)
        user.set_password(password)
        user.save(using=self._db)
        return user

    def create_superuser(self, phone_number, password=None, **extra_fields):
        extra_fields.setdefault('is_staff', True)
        extra_fields.setdefault('is_superuser', True)
        return self.create_user(phone_number, password, **extra_fields)
```

```
class User(AbstractBaseUser, PermissionsMixin):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
```

```

phone_number = models.CharField(max_length=15, unique=True)
first_name = models.CharField(max_length=50, blank=True)
last_name = models.CharField(max_length=50, blank=True)
is_active = models.BooleanField(default=True)
is_staff = models.BooleanField(default=False)
date_joined = models.DateTimeField(default=timezone.now)

objects = UserManager()
USERNAME_FIELD = 'phone_number'
REQUIRED_FIELDS = []

```

```

def __str__(self):
    return self.phone_number

```

II. Facilities App (**facilities/models.py**)

Python

```

import uuid
from django.db import models
from django.utils.text import slugify

class Address(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    address_line = models.CharField(max_length=300)
    area = models.CharField(max_length=100, blank=True)
    city = models.CharField(max_length=100, blank=True)
    district = models.CharField(max_length=100)
    division = models.CharField(max_length=100)
    postal_code = models.CharField(max_length=20, blank=True)
    latitude = models.DecimalField(max_digits=9, decimal_places=6, null=True, blank=True)
    longitude = models.DecimalField(max_digits=9, decimal_places=6, null=True, blank=True)

    class Meta:
        indexes = [models.Index(fields=["district"]), models.Index(fields=["division"])]

class PracticeLocation(models.Model):
    class LocationType(models.TextChoices):
        HOSPITAL = "hospital", "Hospital"
        DIAGNOSTIC_CENTER = "diagnostic_center", "Diagnostic Center"
        CHAMBER = "chamber", "Chamber"

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    location_type = models.CharField(max_length=30, choices=LocationType.choices)
    name = models.CharField(max_length=250)

```

```

branch = models.CharField(max_length=200, blank=True)
slug = models.SlugField(max_length=280, unique=True, blank=True)
address = models.ForeignKey(Address, on_delete=models.PROTECT,
related_name="locations")
phone = models.CharField(max_length=50, blank=True)
email = models.EmailField(blank=True)
logo = models.ImageField(upload_to="facilities/logos/", blank=True, null=True)
image = models.ImageField(upload_to="facilities/images/", blank=True, null=True)
description = models.TextField(blank=True)
tagline = models.CharField(max_length=255, blank=True)
badge = models.CharField(max_length=50, blank=True)
rating = models.FloatField(default=0.0)
reviews_count = models.IntegerField(default=0)
open_timing = models.CharField(max_length=100, blank=True)
is_verified = models.BooleanField(default=False)
is_active = models.BooleanField(default=True)
created_at = models.DateTimeField(auto_now_add=True)

```

```

class Meta:

```

```

    indexes = [models.Index(fields=["location_type"])]
    ordering = ["-created_at"]

```

```

def save(self, *args, **kwargs):

```

```

    if not self.slug:
        b = f"-{self.branch}" if self.branch else ""
        self.slug = slugify(f"{self.name}{b}")
    super().save(*args, **kwargs)

```

```

@property

```

```

def detail(self):
    return getattr(self, f"{self.location_type}_detail", None)

```

```

class HospitalCategory(models.Model):

```

```

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    name = models.CharField(max_length=100)
    slug = models.SlugField(max_length=120, unique=True, blank=True)
    icon = models.CharField(max_length=50, default='Building2')
    description = models.TextField(blank=True)
    count = models.IntegerField(default=0)

```

```

def save(self, *args, **kwargs):

```

```

    if not self.slug:
        self.slug = slugify(self.name)
    super().save(*args, **kwargs)

```

```

class HospitalService(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    name = models.CharField(max_length=150)
    icon = models.CharField(max_length=50, default='Activity')
    description = models.TextField(blank=True)

class Hospital(models.Model):
    location = models.OneToOneField(PracticeLocation, primary_key=True,
on_delete=models.CASCADE, related_name="hospital_detail")
    categories = models.ManyToManyField(HospitalCategory, related_name="hospitals",
blank=True)
    services = models.ManyToManyField(HospitalService, related_name="hospitals",
blank=True)
    has_diagnostic_center = models.BooleanField(default=True)

class DiagnosticCenterCategory(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    name = models.CharField(max_length=150)
    slug = models.SlugField(max_length=170, unique=True, blank=True)
    parent = models.ForeignKey('self', null=True, blank=True, related_name='children',
on_delete=models.CASCADE)
    icon = models.CharField(max_length=100, blank=True)
    description = models.TextField(blank=True)

    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.name)
        super().save(*args, **kwargs)

class DiagnosticService(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    name = models.CharField(max_length=150)
    icon = models.CharField(max_length=50, default='FlaskConical')
    description = models.TextField(blank=True)

class DiagnosticCenter(models.Model):
    location = models.OneToOneField(PracticeLocation, primary_key=True,
on_delete=models.CASCADE, related_name="diagnostic_center_detail")
    categories = models.ManyToManyField(DiagnosticCenterCategory, related_name="centers",
blank=True)
    services = models.ManyToManyField(DiagnosticService, related_name="centers",
blank=True)

```

```

class Chamber(models.Model):
    location = models.OneToOneField(PracticeLocation, primary_key=True,
on_delete=models.CASCADE, related_name="chamber_detail")
    doctor = models.ForeignKey("doctors.Doctor", on_delete=models.CASCADE,
related_name="chambers")
    assistant_phone = models.CharField(max_length=50, blank=True)

```

III. Doctors App (**doctors/models.py**)

Python

import uuid

from django.db import models

from facilities.models import PracticeLocation

from django.utils.text import slugify

```

class DoctorSpecialty(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    name = models.CharField(max_length=100)
    slug = models.SlugField(max_length=120, unique=True, blank=True)
    icon = models.CharField(max_length=50, default='Stethoscope')
    description = models.TextField(blank=True)

```

```

def save(self, *args, **kwargs):
    if not self.slug:
        self.slug = slugify(self.name)
    super().save(*args, **kwargs)

```

```

class Doctor(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    name = models.CharField(max_length=200)
    specialties = models.ManyToManyField(DoctorSpecialty, related_name='doctors')
    qualification = models.TextField()
    experience = models.CharField(max_length=50)

```

```

class DoctorAffiliation(models.Model):
    CONSULTATION_TYPES = [
        ("In-patient", "In-patient"),
        ("Chamber", "Chamber"),
        ("Doctor", "Doctor")
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    doctor = models.ForeignKey(Doctor, on_delete=models.CASCADE,
related_name="affiliations")

```

```

        location = models.ForeignKey(PracticeLocation, on_delete=models.CASCADE,
related_name="affiliations")
        consultation_type = models.CharField(max_length=50, choices=CONSULTATION_TYPES,
default="Chamber")
        fee = models.DecimalField(max_digits=8, decimal_places=2)

class AffiliationSchedule(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    affiliation = models.ForeignKey(DoctorAffiliation, on_delete=models.CASCADE,
related_name='schedules')
    day_of_week = models.CharField(max_length=20)
    start_time = models.TimeField()
    end_time = models.TimeField()

```

IV. Tests App (**tests/models.py**)

Python

```

import uuid
from django.db import models
from django.core.exceptions import ValidationError
from facilities.models import PracticeLocation
from django.utils.text import slugify

```

```

class TestCategory(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    name = models.CharField(max_length=150)
    slug = models.SlugField(max_length=170, unique=True, blank=True)
    icon = models.CharField(max_length=100, blank=True)
    description = models.TextField(blank=True)
    is_active = models.BooleanField(default=True)
    order = models.PositiveIntegerField(default=0)

```

```

class Meta:
    ordering = ['order', 'name']

```

```

def save(self, *args, **kwargs):
    if not self.slug:
        self.slug = slugify(self.name)
    super().save(*args, **kwargs)

```

```

class Test(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    category = models.ForeignKey(TestCategory, related_name='tests',
on_delete=models.CASCADE)

```

```

name = models.CharField(max_length=200)
slug = models.SlugField(max_length=220, unique=True, blank=True)
code = models.CharField(max_length=50, blank=True)
description = models.TextField(blank=True)
sample_type = models.CharField(max_length=100, blank=True)
preparation_instructions = models.TextField(blank=True)
fasting_required = models.BooleanField(default=False)
report_time_hours = models.PositiveIntegerField(null=True, blank=True)
is_active = models.BooleanField(default=True)

def save(self, *args, **kwargs):
    if not self.slug:
        self.slug = f"{slugify(self.name)}-{uuid.uuid4().hex[:6]}"
    super().save(*args, **kwargs)

class FacilityTest(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    location = models.ForeignKey(PracticeLocation, on_delete=models.CASCADE,
related_name="offered_tests")
    test = models.ForeignKey(Test, on_delete=models.CASCADE, related_name="offered_at")
    price = models.DecimalField(max_digits=10, decimal_places=2)
    discounted_price = models.DecimalField(max_digits=10, decimal_places=2, null=True,
blank=True)
    original_price = models.DecimalField(max_digits=10, decimal_places=2, null=True,
blank=True)
    discount = models.CharField(max_length=50, blank=True)
    report_time = models.CharField(max_length=100, blank=True)
    is_available = models.BooleanField(default=True)
    home_sample_collection = models.BooleanField(default=False)
    updated_at = models.DateTimeField(auto_now=True)

class Meta:
    constraints = [
        models.UniqueConstraint(fields=["location", "test"],
name="unique_test_price_per_location"),
    ]

def clean(self):
    if self.location.location_type == PracticeLocation.LocationType.CHAMBER:
        raise ValidationError("Chambers cannot offer lab tests.")

```

V. Bookings App (**bookings/models.py**)

Python

```

import uuid
from django.db import models
from accounts.models import User
from doctors.models import DoctorAffiliation
from tests.models import FacilityTest

class BaseBooking(models.Model):
    class Status(models.TextChoices):
        PENDING = "pending", "Pending"
        CONFIRMED = "confirmed", "Confirmed"
        CANCELLED = "cancelled", "Cancelled"
        COMPLETED = "completed", "Completed"
        NO_SHOW = "no_show", "No show"

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name="%s")
    status = models.CharField(max_length=20, choices=Status.choices,
default=Status.PENDING)
    notes = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True

class DoctorBooking(BaseBooking):
    affiliation = models.ForeignKey(DoctorAffiliation, on_delete=models.CASCADE,
related_name="bookings")
    date = models.DateField()
    slot = models.CharField(max_length=50)
    patient_name = models.CharField(max_length=100)

    class Meta:
        constraints = [
            models.UniqueConstraint(fields=["affiliation", "date", "slot"], name="unique_doctor_slot"),
        ]

class LabBooking(BaseBooking):
    facility_test = models.ForeignKey(FacilityTest, on_delete=models.CASCADE,
related_name="bookings")
    pickup_date = models.DateField()
    patient_name = models.CharField(max_length=100)
    patient_phone = models.CharField(max_length=20)
    address = models.TextField()

```


3. Serializers & API Hygiene

Explicit Serializer Fields: Remove `fields = '__all__'` across the codebase to prevent accidental data leakage.

Python

```
# facilities/serializers.py
```

```
from rest_framework import serializers
```

```
from .models import PracticeLocation
```

```
class PracticeLocationSerializer(serializers.ModelSerializer):
```

```
    detail = serializers.SerializerMethodField()
```

```
    class Meta:
```

```
        model = PracticeLocation
```

```
        fields = [
```

```
            'id', 'location_type', 'name', 'branch', 'slug', 'address',
```

```
            'phone', 'email', 'logo', 'image', 'description', 'tagline',
```

```
            'badge', 'rating', 'reviews_count', 'open_timing', 'is_verified', 'detail'
```

```
        ]
```

```
    def get_detail(self, obj):
```

```
        if obj.location_type == PracticeLocation.LocationType.HOSPITAL:
```

```
            return HospitalDetailSerializer(obj.hospital_detail).data
```

```
        if obj.location_type == PracticeLocation.LocationType.DIAGNOSTIC_CENTER:
```

```
            return DiagnosticCenterDetailSerializer(obj.diagnostic_center_detail).data
```

```
        if obj.location_type == PracticeLocation.LocationType.CHAMBER:
```

```
            return ChamberDetailSerializer(obj.chamber_detail).data
```

```
        return None
```

Validated JSON Pricing Input: Replaces the untyped `JSONField` branching.

Python

```
# tests/serializers.py
```

```
class TestPriceInputSerializer(serializers.Serializer):
```

```
    test_id = serializers.UUIDField(required=True)
```

```
    price = serializers.DecimalField(max_digits=10, decimal_places=2, required=True)
```

```
    original_price = serializers.DecimalField(max_digits=10, decimal_places=2, required=False)
```

Throttling & Pagination (`settings.py`):

Python

```
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 20,
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle'
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '100/day',
        'user': '1000/day',
        'register': '10/hour',
        'login': '20/hour'
    }
}
```

4. Service Layer Extraction

Move complex database attachment out of the serializers.

```
# services/test_pricing.py
from django.core.exceptions import ValidationError
from tests.models import FacilityTest, Test
from facilities.models import PracticeLocation

def attach_tests_to_location(
    *,
    location: PracticeLocation,
    test_category_ids: list[str] = None,
    test_ids: list[str] = None,
    test_prices: dict # Expected format: {"test_uuid": {"price": 500, "original_price": 600}}
):
    if location.location_type == PracticeLocation.LocationType.CHAMBER:
        raise ValidationError("Chambers cannot offer lab tests.")

    tests_to_attach = set()

    # 1. Gather tests by individual IDs
    if test_ids:
        tests_to_attach.update(Test.objects.filter(id__in=test_ids))

    # 2. Gather all tests inside requested categories
    if test_category_ids:
```

```

category_tests = Test.objects.filter(category_id__in=test_category_ids)
tests_to_attach.update(category_tests)

if not tests_to_attach:
    return

objs_to_create = []

# 3. Create the links, enforcing strict pricing (No fake prices)
for test in tests_to_attach:
    price_data = test_prices.get(str(test.id))

    if not price_data or 'price' not in price_data:
        raise ValidationError(f"Missing required price for test: {test.name} (ID: {test.id})")

    objs_to_create.append(
        FacilityTest(
            location=location,
            test=test,
            price=price_data['price'],
            original_price=price_data.get('original_price')
        )
    )

# 4. Bulk insert for high performance
if objs_to_create:
    FacilityTest.objects.bulk_create(objs_to_create)

```